Answers to Reviewer Questions

A. Answer to RAQ2&Rigor4: Manual Check

Motivation and Approach. To expand the scope of manual verification, we have doubled the amount of data to be checked. We randomly select 100 test cases for each mutation and NER system (totaling $100 \times 3 \times 4 = 1200$ test cases). Then, we manually verify whether these 1200 test cases are correct. Our manual verification involves two steps: (1) Two authors independently label the generated test cases, and we measure the inter-rater agreement using Cohen's Kappa coefficient. (2) The two authors discuss any disagreements with a third author to reach a consensus.

Results. Table I presents the experimental results. As shown in the table, most of the generated test cases are correct overall, with accuracy rates ranging from 89.3% to 92.3% for all NER systems across different mutation methods. The Cohen's kappa coefficients across all three mutations (76.97%, 60.17%, and 80.73%) verify the reliability of our evaluation process.

TABLE I: Correctness of Generated Test Cases

NER systems	PER	CES	KPS	Overall
Flair-CoNLL	95.0% (95/100)	92.0% (92/100)	90.0% (90/100)	**92.3% (277/300)**
Flair-Ontonotes	93.0% (93/100)	89.0% (89/100)	91.0% (91/100)	91.0% (273/300)
Azure NER	92.0% (92/100)	88.0% (88/100)	93.0% (93/100)	91.0% (273/300)
AWS NER	92.0% (92/100)	87.0% (87/100)	89.0% (89/100)	89.3% (268/300)
Cohen's Kappa	76.97%	60.17%	80.73%	

Answer to **RAQ2:** Expanding the manual evaluation by doubling the scale, our method maintains its ability to generate a high proportion (an overall average of 90%) of mutated sentences that are both grammatically correct and intuitive.

B. Answer to RAQ3&RBQ7&RCQ3: Repair Performance Comparison

Motivation and Approach. To compare with existing repair methods, we performed a comparative analysis using the TIN method's repair module on the error set identified by our approach (as detailed in Section 5.4).

Results. As shown in Table II, our repair approach consistently outperforms TIN's repair module across four NER systems. The improvement varies from 10.9% for Flair-CoNLL to 29.2% for Azure NER, with an average improvement of 19.5%. Since our method and the TIN approach use different testing perspectives for error detection, they naturally reveal distinct types of errors with unique characteristics. As a result, we argue that repair methods suited for one type of error may not be effective for errors found through other testing approaches.

TABLE II: Comparison of Repair Methods

NER systems	TIN repair	Our Repair	Improvement
Flair-CoNLL	28.1% (41/146)	39.0% (57/146)	+10.9%
Flair-Ontonotes	21.6% (36/167)	38.9% (65/167)	+17.3%
Azure NER	18.9% (44/233)	48.1% (112/233)	+29.2%
AWS NER	19.4% (49/252)	40.1% (101/252)	+20.7%

Answer to **RAQ3&RBQ7&RCQ3:** By applying the TIN repair module to our error set, we observe that the TIN repair method is incompatible with the error patterns identified by our approach.

C. Answer to RBQ2&Rigor1&Rigor4: Number of Generated Test Cases and Per-category Error Detection

Motivation and Approach. In the final paragraph of Section 4.3 in the submission, we present data on generated test cases. Regarding the concern about Rigor4's PER evaluation being under-sampled: Since the PER mutation operator is only applicable to parallel patterns, the total number of generated test cases is limited. Specifically, the range of 13-46 represents the number of suspicious pairs identified from a total of 679 test case pairs.

As pointed out by Reviewer B, showing per-category detection rates would clarify whether the improvement is widespread or limited to a fragile entity type. We conducted an analysis of suspicious entities within suspicious sentence pairs, categorizing them according to their original labels and calculating the proportion of each category. Since different NER systems employ varying classification standards, and our mutation operator, such as CES, filters out certain unsuitable entity types, we unified the granularity by consolidating the categories into four main types according to the CoNLL standard. For the Ontonotes classification standard, we mapped GPE, LOC, and FAC to a unified LOC category, while other types, such as EVENT and WORK_OF_ART, were mapped to the MISC category. For the Azure classification standard, we unified PER and PERSONTYPE under the PER category.

Results. Entity category distributions are generally balanced across systems, with most categories accounting for 20–45% of detections. However, PER entities consistently represent the smallest proportion, ranging from 6.8% to 23.9% across Flair-CoNLL, Flair-Ontonotes, AWS NER, and Azure NER respectively.

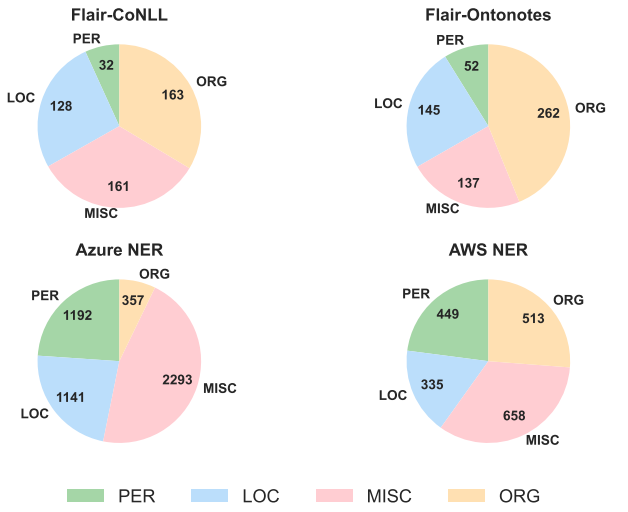

Fig. 1: Entity Type Distribution of Generated Cases.

TABLE III: Total Number of errors detected

NER systems	PER	CES	KPS	Overall
Flair-CoNLL	13	304	112	429
Flair-Ontonotes	13	348	124	485
Azure NER	46	1418	485	1949
AWS NER	36	797	455	1288
Total	108	2867	1176	4151

Answer to **RBQ2&Rigor1&Rigor4:** Entity category distributions remain relatively balanced across systems with no single category dominating excessively.

D. Answer to RBQ3: NER system × operator breakdowns

Motivation and Approach. Precision and repair rate details are provided in Tables 5 and 6. Table 5 presents the true postives, sampling size and total errors, and Table 6 shows repair results based on manual validation of the sampled data. Additional reporting on total errors with NER system × operator breakdowns is given in Table III.

Results. The total errors with NER system × operator breakdowns is given in Table III.

Answer to **RBQ3:** We provide the total errors with NER system × operator breakdowns in III.

E. Answer to RAQ5: Statistical Comparsion Analysis

Motivation and Approach. As pointed out by Reviewer A, we conducted a **hypergeometric** test to evaluate the statistical relationship between the two error sets detected by our method and TIN. The null hypothesis H_0 assumes that the intersection between the two sets occurs randomly and independently.

Results. The test yielded a p-value < 0.05, leading us to reject H_0. With an expected intersection E(X) = 853.9 under random assumption, our observed overlap of 209 falls in the left tail, indicating that the two sets are significantly negatively correlated. This suggests that errors detected by our method and TIN tend to occur in different directions, which also reflect that the different perspective of our method works.

Answer to **RAQ5:** The error detected by our method and TIN are significantly negatively correlated, which demonstrates that our method systematically detects different categories of errors from alternative perspectives.

F. Answer to RCQ4: HELP AND HARM

Motivation and Approach. As pointed out by Reviewer C: Showing both "help" and "harm" rates would better illustrate the practical significance of the repair module.

Our repairs are carried out on the inconsistent original-mutated pairs that we identified. Therefore, from their perspective, some outcome is considered "help"—the inconsistent pairs are corrected to become consistent (and manually verified as correct), which forms the basis for the repair success rate reported in RQ4. No outcome is considered "harm" since consistent pairs are not modified. However, at a more detailed level, some inconsistent original-mutated pairs initially included one correct and one incorrect NER result, but after repair, the correct result became incorrect. These cases are regarded as repair failures (i.e., 1-repair success rate) in RQ4. We calculated the number of this scenario on the four tested NER systems.

Results. We calculated the number of this detailed scenario on the four tested NER systems. The results show 110, 114, 53, and 57 cases on Azure NER, AWS NER, Flair-CoNLL, and Flair-Ontonotes respectively. The calculation was based on the error set from RQ4, comprising 798 original sentences and their corresponding mutated versions, totaling 1,596 sentences.

Answer to **RCQ4:** We think that our repairs provide "help" with no "harm" in our scenario since the repair is conducted at pair level and only inconsistent pairs are modified, but from a detailed level, we obtained data showing 110, 114, 53, and 57 cases across four NER systems where correct results are modified to become incorrect.

G. Answer to RAQ6: strong LLM baselines

Motivation and Approach. As pointed out by Reviewer A, due to the advancements in large language model technology, various commercial large language models have become widely adopted for named entity recognition tasks, making it pertinent to assess their efficacy in entity classification consistency. The prompt we use is displayed as in 2. We provide an example in **??**.

Results. To investigate this, we randomly selected 3,000 test case pairs and applied our method on one of the most popular

```
    prompt = f"""
Please perform entity recognition on the following two sentences according to AWS entity classification standards:
Entity Category Definitions:
- COMMERCIAL_ITEM: Brand products
- DATE: Complete dates (e.g., 11/25/2017), days of the week (Tuesday), months (May), or times (8:30 AM)
- EVENT: Events such as holidays, concerts, elections, etc.
- LOCATION: Specific locations such as countries, cities, lakes, buildings, etc.
- ORGANIZATION: Large organizations such as governments, companies, religions, sports teams, etc.
- PERSON: Individuals, groups of people, nicknames, fictional characters
- QUANTITY: Quantified amounts such as currency, percentages, numbers, bytes, etc.
- TITLE: Official names of creative works such as movies, books, songs, etc.
- OTHER: Entities that do not belong to any of the above categories

Original sentence: {original_sentence}
Mutated sentence: {mutated_sentence}

Please identify entities in both sentences respectively and output strictly in the following JSON format:
{{
  "original": {{
    "COMMERCIAL_ITEM": [{{"text": "entity text", "start": start_position, "end": end_position}}],
    "DATE": [{{"text": "entity text", "start": start_position, "end": end_position}}],
    "EVENT": [{{"text": "entity text", "start": start_position, "end": end_position}}],
    "LOCATION": [{{"text": "entity text", "start": start_position, "end": end_position}}],
    "ORGANIZATION": [{{"text": "entity text", "start": start_position, "end": end_position}}],
    "PERSON": [{{"text": "entity text", "start": start_position, "end": end_position}}],
    "QUANTITY": [{{"text": "entity text", "start": start_position, "end": end_position}}],
    "TITLE": [{{"text": "entity text", "start": start_position, "end": end_position}}],
    "OTHER": [{{"text": "entity text", "start": start_position, "end": end_position}}]
  }},
  "mutated": {{
    "COMMERCIAL_ITEM": [{{"text": "entity text", "start": start_position, "end": end_position}}],
    "DATE": [{{"text": "entity text", "start": start_position, "end": end_position}}],
    "EVENT": [{{"text": "entity text", "start": start_position, "end": end_position}}],
    "LOCATION": [{{"text": "entity text", "start": start_position, "end": end_position}}],
    "ORGANIZATION": [{{"text": "entity text", "start": start_position, "end": end_position}}],
    "PERSON": [{{"text": "entity text", "start": start_position, "end": end_position}}],
    "QUANTITY": [{{"text": "entity text", "start": start_position, "end": end_position}}],
    "TITLE": [{{"text": "entity text", "start": start_position, "end": end_position}}],
    "OTHER": [{{"text": "entity text", "start": start_position, "end": end_position}}]
  }}
}}

If a category has no entities, please return an empty array []. Only return the JSON format result without adding any additional explanatory text.
"""
```

Fig. 2: Prompt of test on LLM.

large models, GPT-3.5, for NER error detection. Since different NER systems employ varying classification standards, we adopted the AWS NER classification standard as a compromise and obtained classification results by calling the OpenAI "gpt-3.5-turbo" API, discovering a total of 181 errors.

Answer to **RAQ6:** We randomly chose 3,000 original-mutated pairs from our dataset to test the LLM. The results showed that our method detected over 180 inconsistent pairs, proving that our testing approach can still effectively identify errors in LLMs.